

# Programming Assignment 1

Filename: pa1.c

You must include the following commented lines at the beginning of your code to declare authorship. Submissions that do not include these lines will receive a 10% deduction.

```
/* COP 3502C PA1  
This program is written by: Your Full Name */
```

In this activity, you will practice dynamic memory allocation in C, focusing on the following:

- Allocating memory for arrays of structures, structure pointers, integers, and strings
- Manipulating pointers to access and modify dynamically allocated data
- Working with structures that have dynamically allocated arrays of structures and arrays of integers
- Properly freeing dynamically allocated memory to prevent memory leaks
- Writing functions that handle queries according to the given specifications

## Background

A cat shelter (i.e., store) has been steadily growing and needs to upgrade its data systems. Currently, the shelter relies on an outdated method for managing its information. To address this, they want to develop a centralized online system. You are tasked with creating a program that does the following:

- Track the number of breeds being stored.
- Store the maximum capacity for each breed per kennel.
- Monitor each kennel's maximum capacity and ensure it satisfies the constraints for transferring cats across kennels.
- Store information for each individual cat.
- Perform queries on the stored data.

## Variables and Context

The program you are writing manages a single cat store that has several kennels across different locations. Each kennel maintains a set of cats residing in it, along with its current occupancy and maximum capacity. Each cat's information is stored in the program, including its name, age, weight, breed, and status.

The program also maintains a dictionary, which is a list of strings representing the different cat breeds that may reside at any of its locations.

One piece of information stored and managed for the store is a set of capacity constraints for each kennel and breed type. This information is represented as a 2-dimensional array, where the number of rows corresponds to the number of kennels and the number of columns corresponds to the number of breeds in the dictionary. A kennel's maximum capacity is calculated as the sum of its individual breed capacities.

For example, assume there are 4 breeds and 5 kennels. Further, kennel #2 has the following capacities:

```
capacities[2] = {4, 2, 1, 5}
```

This means that kennel #2 can have a maximum of 5 cats of breed type 3. The overall maximum capacity of kennel #2 is 12, which is the sum of its individual breed capacities.

Although a kennel's maximum capacity is tracked, it does not need to be completely filled when populated. Therefore, an occupancy variable is used to keep track of the actual number of cats residing in a kennel. The occupancy must never exceed the kennel's maximum capacity.

One piece of information tracked for each cat is its current status. In the Cat struct, the status field is stored as an integer with possible values 0, 1, or 2. This integer serves as an index into the global constant array CAT\_STATUS. Each cat's status must be initialized to 2 (i.e., "AVAILABLE") when first inserted into the system.

The CAT\_STATUS array is an array of strings (defined in the global constant variable section) representing the possible statuses of a cat. For example, if a cat's status value is 0, then its status string can be retrieved by accessing CAT\_STATUS[0], which corresponds to "ADOPTED".

Lastly, the cat's breed is stored as a char \* that should not be dynamically allocated for each cat. Instead, it must point directly to the corresponding string stored in the dictionary.

The remaining members of the structs are self-explanatory and are provided in the required sections below.

## Flow of the Program

The program has mainly two parts: Initialization and Query Processing.

### Initialization

You will be given all the initial information related to the store, kennels, and cats. You should create the structs and store those values and information accordingly. Note that the first set of information is guaranteed not to violate any capacity constraints of the kennels. That is, during this stage, all cat information given can be stored at the kennel where we are attempting to store them. No printing should be done during this stage.

### Query Processing

Next, you will be asked to process several queries about this store. Those are explained in detail later in this document. It is important to note that these queries may or may not violate the constraints, which you should check before performing any data transfer between kennels. Additionally, since you are a DMA expert in C and will not waste memory, you need to properly resize arrays when needed. Properly keep track of pointers, so as not to create duplicates for the same cat, and so on. Printed output depends on the query type and is explained in detail later in this document.

## Required Data Structures

You must use the following structures in this assignment. You are allowed to declare more structures if needed. However, you are not allowed to change the required structures.

```
typedef struct Cat_s {
    char *name;           // Cat's name (dynamic string)
    int age;              // Cat's age
    float weight;         // Cat's weight
    char *breed;          // Cat's breed (already allocated dynamic string)
    int status;           // Cat's status 0, 1, or 2 (default)
} Cat;

typedef struct Kennel_s {
    char *location;       // Kennel's location name (dynamic string)
    int occupancy;        // Actual number of cats currently residing
    Cat **cats;           // The cats residing in the kennel
                        // (dynamic array of pointers to struct)
    int maxCapacity;      // The maximum number of cats that can reside
} Kennel;

typedef struct CatStore_s {
    Kennel *kennels;      // Kennels owned by the store (dynamic array)
    int numKennels;       // Number of kennels owned
    int **capacities;     // Dynamic 2D array indicating the constraints
                        // across kennels and breeds
} CatStore;
```

## Global Constant Variable

The following must be declared in your code:

```
const char CAT_STATUS[3][20] = {"ADOPTED", "PENDING", "AVAILABLE"}
```

## Starter Code

Your task is to complete the given C program. You can access the starter code [here](#).

**Important:** Do not modify the body of the main() function.

## Required Functions

You must implement and use the following functions with the specified prototypes in your solution. You are free to define additional functions as needed (and are recommended to do so).

**Important:** The 14 required functions must be implemented exactly as specified. Therefore, their prototypes must not be modified.

`char **readBreeds(FILE *ifile, int *count)`

### Preconditions

- `ifile` is not NULL and refers to an open file stream in read mode, positioned at the point where the number of breeds is stored
- `count` is not NULL and points to a valid integer variable

### Postconditions

- Returns a dynamically allocated array of dynamically allocated strings containing the breed names read from the file
- `*count` is updated to reflect the number of breeds read
- The input file stream is advanced past the breed data

`char *lookupBreed(char **dictionary, int breedCount, char *breedName)`

### Preconditions

- `dictionary` is a valid array of strings storing breed names
- `breedCount` is a positive integer representing the number of breeds in dictionary
- `breedName` is a valid, non-NULL string representing the breed to search for

### Postconditions

- Returns a pointer to the matching breed string in dictionary if found
- Returns NULL if no matching breed exists

`CatStore *createStore(FILE *ifile, int kennelCount, char **dictionary, int breedCount)`

### Preconditions

- `ifile` is not NULL and refers to an open file stream in read mode, positioned at the start of the store data
- `kennelCount` is a positive integer representing the number of kennels in the store
- `dictionary` is a valid array of strings storing breed names
- `breedCount` is a positive integer representing the number of breeds in dictionary

### Postconditions

- Returns a pointer to a dynamically allocated `CatStore` populated with data read from the file
- All fields of the returned `CatStore` are fully initialized
- The input file stream is advanced past the store data

```
Kennel *createKennels(FILE *ifile, int kennelCount, char **dictionary, int breedCount,
int **constraints)
```

#### **Preconditions**

- ifile is not NULL and refers to an open file stream in read mode, positioned at the start of the kennel data
- kennelCount is a positive integer representing the number of kennels
- dictionary is a valid array of strings storing breed names
- breedCount is a positive integer representing the number of breeds in dictionary
- constraints is a valid 2D array with kennelCount rows and breedCount columns that represents the breed capacity for each kennel

#### **Postconditions**

- Returns a dynamically allocated array of Kennel structures of size kennelCount
- Each kennel is fully populated with location data, cats, and computed capacity values
- The input file stream is advanced past the kennel data

```
Cat **createCats(FILE *ifile, char **dictionary, int breedCount, int catCount)
```

#### **Preconditions**

- ifile is not NULL and refers to an open file stream in read mode, positioned at the start of the cat data
- dictionary is a valid array of strings storing breed names
- breedCount is a positive integer representing the number of breeds in dictionary
- catCount is a non-negative integer representing the number of cats to be created

#### **Postconditions**

- Returns a dynamically allocated array of Cat pointers of size catCount
- Each pointer refers to a dynamically allocated and properly initialized Cat
- The input file stream is advanced past the cat data

```
Cat *createSingleCat(FILE *ifile, char **dictionary, int breedCount)
```

#### **Preconditions**

- ifile is not NULL and refers to an open file stream in read mode, positioned at the start of a single cat's data
- dictionary is a valid array of strings storing breed names
- breedCount is a positive integer representing the number of breeds in dictionary

#### **Postconditions**

- Dynamically allocates and initializes a single Cat
- The cat's status is initialized to the value that corresponds to AVAILABLE
- The breed pointer references the corresponding string in dictionary
- Returns a pointer to the allocated Cat
- The input file stream is advanced past the cat data

```
int canMoveTo(CatStore *s, char *location, char *breed, char **dictionary, int breedCount)
```

**Preconditions**

- s points to a valid populated CatStore
- location is a valid, non-NULL string identifying the destination kennel
- breed is a valid, non-NULL string representing the cat's breed
- dictionary is a valid array of strings storing breed names
- breedCount is a positive integer representing the number of breeds in dictionary

**Postconditions**

- Returns 1 if the cat can be moved to the specified location
- Returns 0 otherwise
- A cat of a given breed can only be moved to the location if the location's maximum capacity is not yet exceeded AND the breed's capacity of this location is not yet exceeded

```
Kennel *getKennelByCat(CatStore *s, Cat *cat)
```

**Preconditions**

- s points to a valid populated CatStore
- cat points to a valid Cat

**Postconditions**

- Returns a pointer to the kennel where cat currently resides
- Returns NULL if the cat is not found

```
int getCatIndex(Kennel *home, Cat *cat)
```

**Preconditions**

- home points to a valid populated Kennel
- cat points to a valid Cat

**Postconditions**

- Returns the index of cat within the kennel's cat array
- Returns -1 if the cat is not found

```
Cat *getCatByName(CatStore *s, char *catName)
```

**Preconditions**

- s points to a valid populated CatStore
- catName is a valid, non-NULL string

**Postconditions**

- Returns a pointer to the cat whose name matches catName
- Returns NULL if no such cat exists

```
void removeCatFromKennel(Kennel *k, Cat *cat)
```

**Preconditions**

- k points to a valid populated Kennel
- cat points to a valid Cat currently residing in k

**Postconditions**

- Removes cat from the kennel's cat array
- Remaining elements are shifted to fill the gap
- The Cat itself is not freed

```
void runQueries(FILE *ifile, CatStore *s, char **dictionary, int breedCount, int numQueries)
```

**Preconditions**

- ifile is not NULL and refers to an open file stream in read mode, positioned at the start of the query data
- s points to a valid populated CatStore
- dictionary is a valid array of strings storing breed names
- breedCount is a positive integer representing the number of breeds in dictionary
- numQueries is a non-negative integer

**Postconditions**

- Processes each query and applies the corresponding operation to the store
- The input file stream is advanced past the query data

```
void freeBreeds(char **dictionary, int breedCount)
```

**Preconditions**

- dictionary is a valid array of strings storing breed names
- breedCount is a positive integer representing the number of breeds in dictionary

**Postconditions**

- Frees all dynamically allocated strings in dictionary
- Frees the dictionary array itself

```
void freeStore(CatStore *store)
```

**Preconditions**

- store points to a valid populated CatStore

**Postconditions**

- Frees all dynamically allocated memory associated with the store

## Query Processing

After storing all data into the data structures, queries will need to be processed based on the inputs.

### Query Type 1

This query prints all information about cats of a given breed, namely their name, weight, age, location, and status.

### Query Type 2

This query updates the status of a specific cat given its name. If the cat's new status is adopted, the cat must be entirely removed from the system, and the cat array must be adjusted accordingly. Remember to free any allocated space.

### Query Type 3

This query attempts to move a cat from one kennel to another. If the transfer is valid based on capacity constraints, the cat is removed from the source kennel, the source kennel's cats array is resized to one fewer cat, the destination kennel's cats array is resized to one more, and the moved cat is added to the destination kennel's cats array. Occupancies are adjusted accordingly. Note: a transfer is only valid if it doesn't exceed any of the maximum capacity or breed-specific capacity constraints.

#### Important Note

This dataset can support a wide range of interesting queries, such as finding all available cats, identifying overcrowded kennels, or determining adoption rates by breed. In fact, we initially included several more complex and engaging queries. However, we realized that the assignment was becoming too lengthy. Therefore, we've decided to limit the scope to just Queries 1-3, as the primary focus of this assignment is dynamic memory allocation.



## Input

The program reads input from a file named `store.txt`. The first line contains a single positive integer  $n$  ( $1 \leq n \leq 10$ ), representing the number of cat breeds. The next  $n$  lines each contain a single string `breedname` of maximum length 25, representing the name of a cat breed that may reside in the store.

```
n
<breedname_1>
<breedname_2>
...
<breedname_n>
```

The next line of input contains a single positive integer  $k$  ( $1 \leq k \leq 100$ ), representing the number of kennels (i.e., locations) in the store. The next  $k$  lines each contain  $n$  integers representing the capacity constraints for each breed at a given kennel, as described in the background section. The maximum overall capacity of a kennel is computed as the sum of all breed capacities on its corresponding line.

```
k
<capacity_0> <capacity_1> <capacity_2> ... <capacity_n>
<capacity_0> <capacity_1> <capacity_2> ... <capacity_n>
...
<capacity_0> <capacity_1> <capacity_2> ... <capacity_n>
```

For each kennel, additional input is provided describing the kennel's location, the number of cats currently housed there, and information about each cat. This format is repeated  $k$  times.

Each kennel description begins with a single line containing a unique string `location` (maximum length 25, guaranteed unique), followed by an integer  $c$  ( $0 \leq c \leq 1,000$ ), representing the number of cats currently in the kennel. The next  $c$  lines contain information for each cat.

Each cat record consists of a string `name` (maximum length 25, guaranteed unique), an integer `age`, a floating-point value `weight`, and a string `breed`. All cats are initialized with a status value that corresponds to "AVAILABLE".

```
<location_1> <c1>
<name_1> <age_1> <weight_1> <breed_1>
<name_2> <age_2> <weight_2> <breed_2>
...
<name_c1> <age_c1> <weight_c1> <breed_c1>
<location_2> <c2>
<name_1> <age_1> <weight_1> <breed_1>
<name_2> <age_2> <weight_2> <breed_2>
...
<name_c2> <age_c2> <weight_c2> <breed_c2>
...
<location_k> <ck>
<name_1> <age_1> <weight_1> <breed_1>
<name_2> <age_2> <weight_2> <breed_2>
...
<name_ck> <age_ck> <weight_ck> <breed_ck>
```

After all kennel data has been read, the next line contains a single positive integer  $q$  ( $1 \leq q \leq 100$ ), representing the number of queries to process. Each of the next  $q$  lines begins with an integer in the range 1–3, indicating the query type. The remaining input on the line depends on the query type.

### Query Type 1

```
1 <breed>
```

<breed> is a string representing a breed name. It is guaranteed that the breed exists in the dictionary; however, there may be no cats of that breed currently in the store.

### Query Type 2

```
2 <num> <name>
```

<num> is an integer in the range 0–2 representing the new status of the cat. <name> is the name of the cat whose status is being updated. It is guaranteed that a cat with this name exists in the store.

### Query Type 3

```
3 <name> <location>
```

<name> is the name of the cat to be moved, and <location> is the destination kennel. It is guaranteed that a cat with this name and a kennel with this location exists in the store.

### Important Note

You may assume that all input strings contain no whitespace.

## Output

The program's output depends on the query type.

### Query Type 1

For every cat matching the requested breed, print the cat's information on a single line in the order the cats were added to the store, using the following format, followed by a newline:

```
<name> <weight> <age> <location> <status>
```

Where:

<name> represents the name of the cat.

<weight> represents the weight of the cat, formatted to 2 decimal places.

<age> represents the age of the cat.

<location> represents the location of the kennel where the cat resides.

<status> represents the cat's status, which is either "PENDING" or "AVAILABLE".

If no cat of the requested breed exists in the store, print the following message on a single line, followed by a newline:

```
No cat with breed <breed>
```

### Query Type 2

Print a single line in the following format, followed by a newline:

```
<name> is now <status>!
```

Here, <name> is the name of the cat being updated, and <status> is one of "ADOPTED", "PENDING", or "AVAILABLE", depending on the input. The global array CAT\_STATUS must be used to generate the status string.

### Query Type 3

If the cat can be successfully moved to the requested location, print the following message, followed by a newline:

```
<name> moved successfully to <location>
```

Otherwise, print the following message, followed by a newline:

```
<location> cannot take a <breed> cat!
```

## Sample Input store.txt

```
5
Ragdoll
Maincoone
Siamese
Tuxedo
Tabby
3
12 5 10 6 4
18 17 13 6 9
0 5 3 20 5
Polterland 4
Charlie 12 52.55 Ragdoll
Caesar 2 512.2 Siamese
Cleo 4 8.6 Tuxedo
Patra 4 11.15 Ragdoll
Auburndale 2
Coco 14 23.2 Ragdoll
Channel 13 18.3 Ragdoll
Narnia 1
Sad 18 92.00 Tabby
9
1 Ragdoll
2 1 Charlie
2 0 Charlie
1 Ragdoll
3 Caesar Auburndale
3 Sad Narnia
3 Coco Narnia
2 0 Cleo
1 Tuxedo
```

## Sample Output

```
Charlie 52.55 12 Polterland AVAILABLE
Patra 11.15 4 Polterland AVAILABLE
Coco 23.20 14 Auburndale AVAILABLE
Channel 18.30 13 Auburndale AVAILABLE
Charlie is now PENDING!
Charlie is now ADOPTED!
Patra 11.15 4 Polterland AVAILABLE
Coco 23.20 14 Auburndale AVAILABLE
Channel 18.30 13 Auburndale AVAILABLE
Caesar moved successfully to Auburndale
Sad moved successfully to Narnia
Narnia cannot take a Ragdoll cat!
Cleo is now ADOPTED!
No cat with breed Tuxedo
```

## Additional Implementation Requirements

- You may only use malloc, calloc, or realloc for memory allocation
- Ensure all required function prototypes are implemented within your program.
- Only one instance of each cat should be created, and their pointers should be used consistently throughout the program. In other words, there should be only one malloc call for each cat
- Your code design must not use triple pointers (\*\*\*). You should limit the pointer usage to single or double pointers for simplicity and clarity
- For strings, you must first read the input into a static char array, then allocate the exact amount of memory needed based on its length and copy the string using strcpy function as demonstrated in class.
- You must free all the memory to receive full credit.
- You are not allowed to write a function that will receive or return a whole structure (i.e., only an address or pointer to a struct is allowed)
- Make sure not to use any malloc/calloc/realloc for cat's breed property. It should only point to an existing breed in the dictionary
- Your code must have no memory leaks at the end of its execution.
- Only one instance of each breed should be created in the dictionary, and their pointers should be used consistently throughout the program. In other words, there should only be one malloc call for each breed string. Cat breed pointers should point to the dictionary entry, not allocate new memory
- Make sure your file name is pa1.c.
- Make sure to test your code thoroughly using your own test cases in addition to the provided ones. During grading, we will run additional test cases to evaluate your code. Therefore, passing all sample test cases does not guarantee full credit if your code fails the grading test cases. Such failures indicate that your code contains bugs or does not handle certain situations described in the assignment properly

## Hints

- Before starting your code, draw out the program's logic and scenario to help with its implementation.
- Make sure to test your code gradually as you implement functions to avoid complex debugging later.
- Update elements as needed to maintain proper functionality.
- Consider how the 2D-array capacities relates to each kennel and what its role when validating a transfer.
- Make sure you have a pretty good idea of DMA based on the lecture and notes before starting.

## Tentative Rubric (Subject to Change)

A set of test cases will be used to evaluate whether your program produces the expected output. Each failed test case will result in a grade deduction per the rubric below. In addition to the sample test cases, we will use additional (hidden) test cases during grading. Therefore, passing the sample test cases does not guarantee full credit. You are strongly encouraged to thoroughly test your code against a variety of inputs. Because some test cases are hidden, your final score will not be visible until the submission has been fully graded.

**Important:** Any attempt to hard-code the expected output or otherwise circumvent the grading process constitutes a violation of academic integrity and will be handled in accordance with university policy.

- If your code does not compile on Gradescope, you will receive a score of 0.
- If you modify, omit, or fail to use the required data structures, you may receive a score of 0.
- Failure to use dynamic memory allocation for storing data will result in a score of 0.
- Correct implementation of all required functions and specifications: 35%.
- Properly freeing all dynamically allocated memory with zero memory leaks (assuming all required `malloc` calls are implemented): 15%.
- Passing all test cases with exact output matching (approximately 10 test cases, including the provided sample test cases): 50%.

## Clarifications

For questions or clarifications about the program requirements, please post them in the corresponding discussion thread on Webcourses. If additional clarification is needed, you are welcome to visit the TAs or the instructor during office hours. Requests for help with debugging should be brought to office hours rather than sent by email.